MAE 204: Final Project Report

Jobanpreet Mutti
Mechanical Engineering
jmutti@ucsd.edu

Dhanush Srikanth
Mechanical Engineering
dsrikanth@ucsd.edu

March 18, 2026

1 Report Summary

The files created for this project serve to facilitate the movement of the youBot mobile robot to successfully grab and move a cube. Fundamentally, the approach relies on separating the problem into the following logical components: state prediction, trajectory generation, and feedback control. Through this division, the system can reliably move the end-effector through a sequence of eight trajectory segments. The ScrewTrajectory function was utilized to generate these reference configurations, which ensured smooth, rest to rest motion between the waypoints. The generated reference trajectory provides the desired path of the robot.

However, the desired configurations are meaningless without the ability to properly execute and course correct. The feedforward and PI feedback control law bridges this gap. The feedforward component anticipates the necessary motion to follow the trajectory, the P component reacts to instantaneous pose errors, while the I feedback actively corrects for accumulated positional and orientational errors. To translate these end-effector commands into physical robot motion, the code utilizes the pseudoinverse of the mobile manipulator Jacobian. This is how the software decides responsibility between the four Mecanum wheels and the five arm joints. the pseudoinverse optimally determines the joint and wheel speeds required to achieve the desired end-effector twist. The following equations were used:

$$\mathbf{V} = \mathbf{V}_d + K_p \mathbf{X}_{err} + K_i \int \mathbf{X}_{err}\, dt \tag{1}$$

$$\dot{\mathbf{q}} = J^{\dagger}\mathbf{V} \tag{2}$$

For simulating the movement of the youBot, the kinematics rely on first order Euler integration to increment the robot's configuration at each time step. While the foundational requirements of the simulation were met, a critical physical constraint had to be managed. In a purely mathematical simulation, wheel and joint velocities could approach infinity. To ground the results in reality, the NextState function enforces strict magnitude limits on the commanded velocities. This prevents physically impossible movements.

The result is a simulation system that not only sets a clear path, but also has the capacity to produce the corrective feedback needed to achieve it with realistic limitations.

2 Results

2.1 Best Case

2.1.1 Brief Description of Best Case

The best case relies on a feedforward and a proportional controller (feedforward-plus-P). The best case does also have a PI control built in, however all the gains in the K_i matrix are set to zero. To achieve the best case, K_p was simply set to the 6 by 6 identity matrix.

Initially, the chassis orientation was set to .6 rad. This ensured that the end-effector position error was greater than .2 meters, and that the orientation error was greater than 30° when compared to the desired initial end-effector pose. Specifically, the initial position error was .3995 meters and the initial orientation error was .6231 rad.

2.1.2 Video of Best Case

The video for the best case can be found here: `https://drive.google.com/file/d/1XoF7mYHcFBwHeTWI1AUr1x_V7Nx3rNLj/view?usp=sharing`

2.1.3 Plots of Best Case

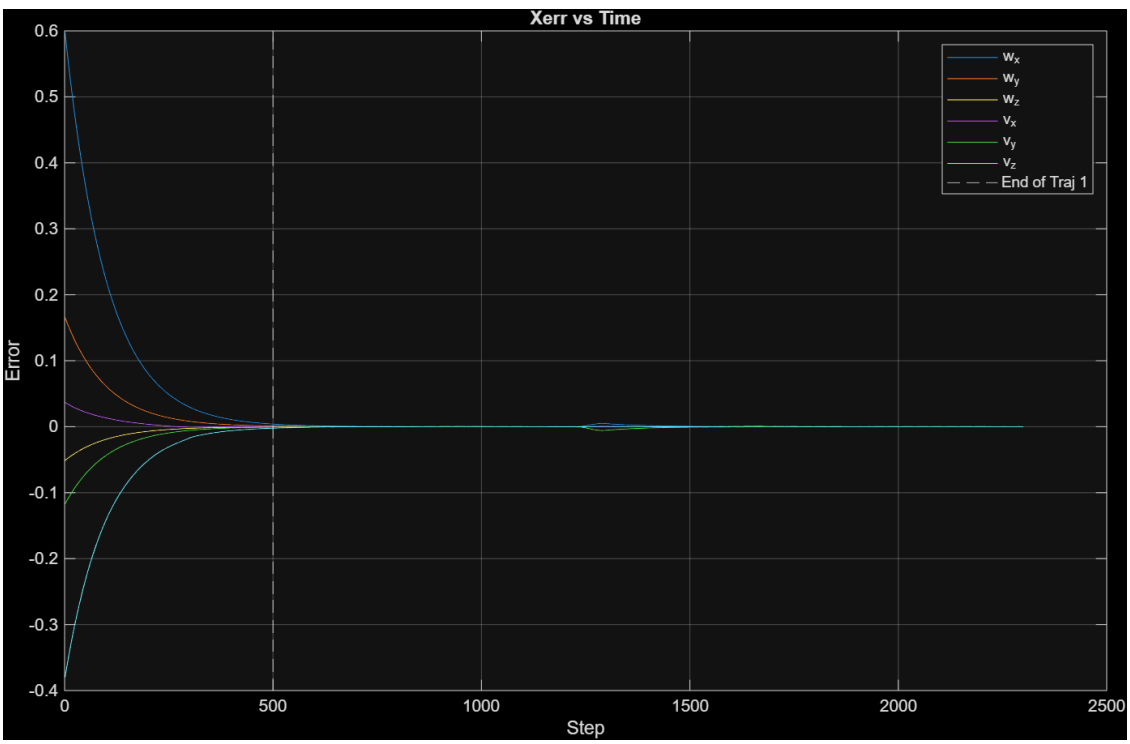

Figure 1: Xerr vs. Time for Best Case

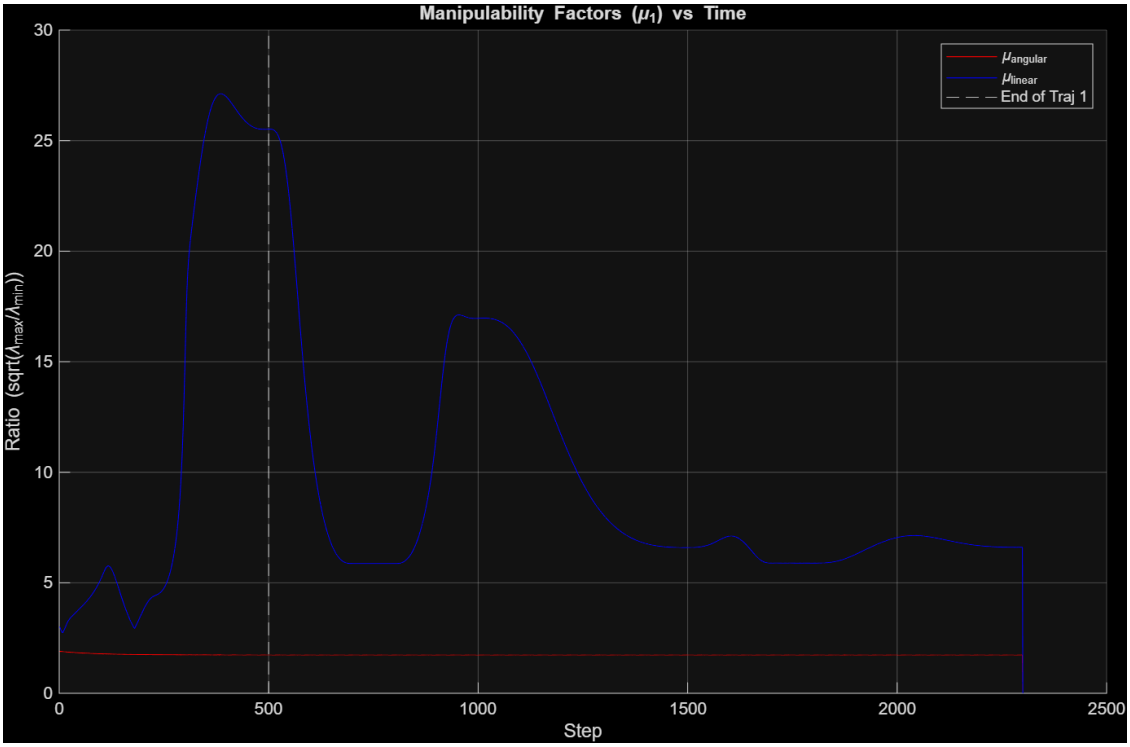

Figure 2: $u_1(A_w)$ and $u_1(A_v)$ vs. Time for Best Case

2.2 Overshoot Case

2.2.1 Brief Description of Overshoot Case

To showcase overshoot behavior, a feedforward-plus-PI controller was implemented. The controller ultimately had the same core structure as the Best Case; however, the integral gains matrix K_i was set to the 6 by 6 identity matrix. K_p remained the same as in the Best Case.

If set well, the addition of K_i allows the controller to eliminate long-term errors. However, this property was used here to create an underdamped response, as the controller attempted to correct the accumulated error and consequently overshot.

Right before the end of the trajectory one segment, the error was eliminated. However, Figure 3 reveals that the overshoot behavior caused some of the error to return immediately afterward. It was found that with other values of K_i and K_p, it was quite easy to completely eliminate the overshoot by the end of trajectory one and thereafter. However, those values were not used, as the overshoot behavior was difficult to observe visually in CoppeliaSim. For the sake of completeness, diagonal gains of 2 for K_p and diagonal gains of 1.5 for K_i resulted in initial overshoot behavior that was almost fully eliminated by the end of trajectory one. The Xerr for this extra case can be seen in Figure 5.

Initially, the chassis orientation was set to .6 rad. This ensured that the end-effector position error was greater than .2 meters, and that the orientation error was greater than 30° when compared to the desired initial end-effector pose. Specifically, the initial position error was .3995 meters and the initial orientation error was .6231 rad.

2.2.2 Video of Overshoot Case

The video for the Overshoot case can be found here: `https://drive.google.com/file/d/1O1Bo-_wMZ0FZnmW4ZM8LGzwAz7YRDTcx/view?usp=sharing`

2.2.3 Plots of Overshoot Case

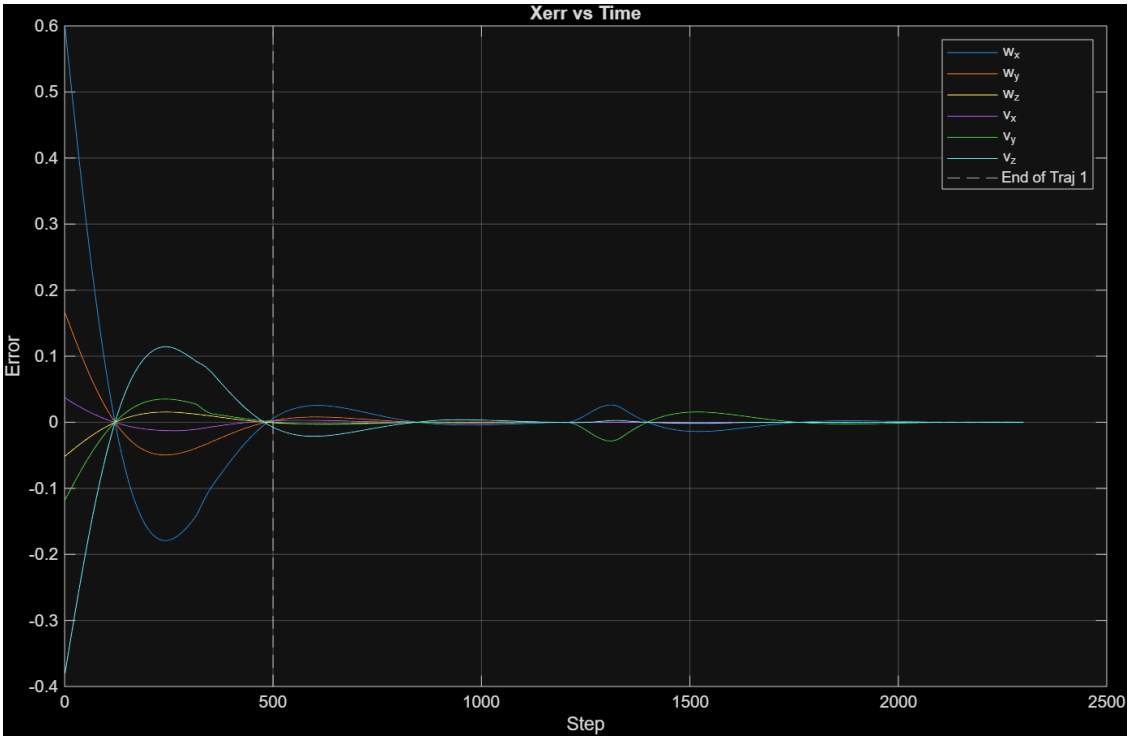

Figure 3: Xerr vs. Time for Overshoot Case

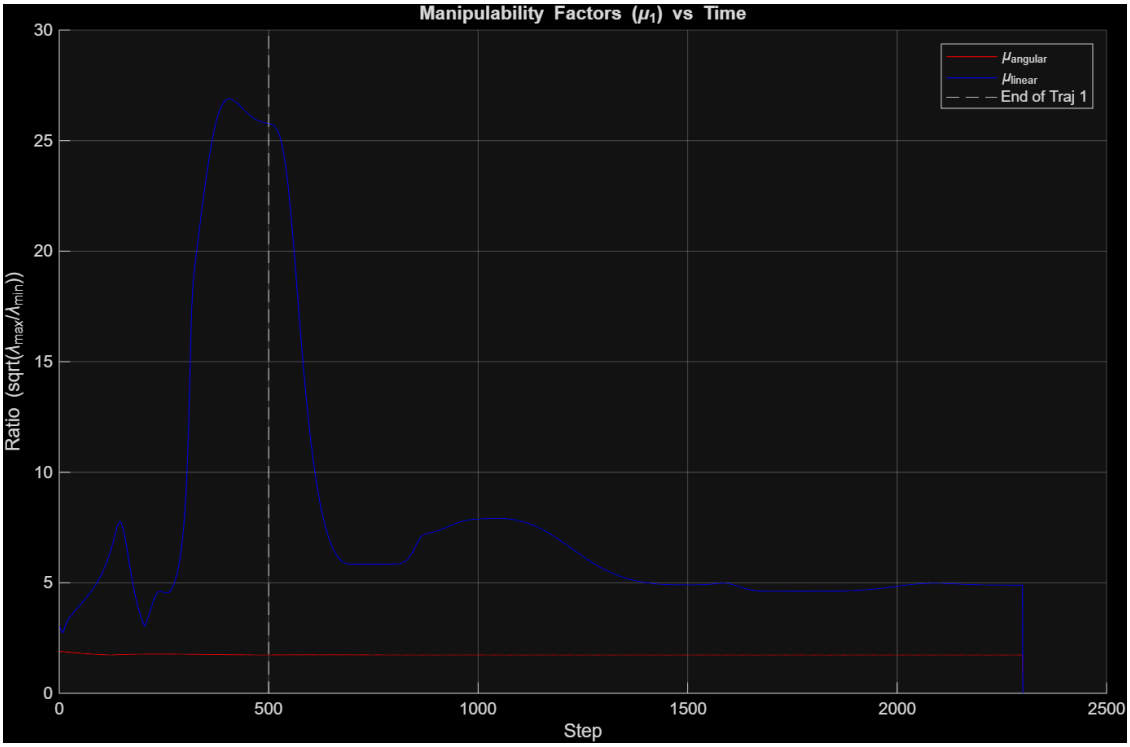

Figure 4: $u_1(A_w)$ and $u_1(A_v)$ vs. Time for Overshoot Case

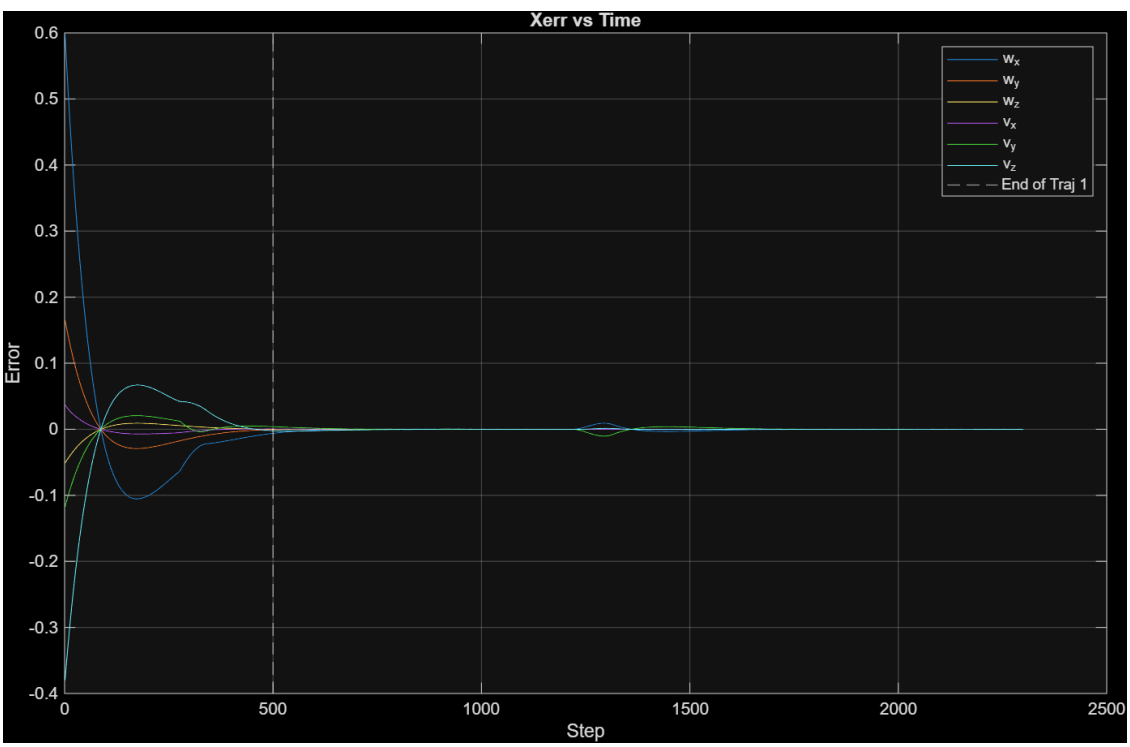

Figure 5: Extra Xerr vs. Time for Overshoot Case with $K_p = 2I$ and $K_i = 1.5I$

2.3 New Task Case

2.3.1 Brief Description of New Task Case

The New Task case relies on a feedforward and a proportional controller (feedforward-plus-P). The New Task case does have a PI control built in, however all the gains in the K_i matrix are set to zero. To finish the new task, K_p was simply set to $2I$. As can be seen in Figure 6, this configuration effectively removed all error well before the end-effector reached the end of the first trajectory segment.

For the new task, the initial and final position of the cube were altered. The cube was initially located at x = .8 m, y = .3 m, z = .025m. The cube's orientation was initially aligned with the world frame. The cube's final destination was set to x = 1 m, y = -.2 m, z = .025 m. The cube's final orientation was obtained by rotating it 90 degrees clockwise around the z axis.

Initially, the chassis orientation was set to .6 rad. This ensured that the end-effector position error was greater than .2 meters, and that the orientation error was greater than 30° when compared to the desired initial end-effector pose. Specifically, the initial position error was .3995 meters and the initial orientation error was .6231 rad.

2.3.2 Video of New Task Case

The video for the New Task case can be found here: `https://drive.google.com/file/d/1QPLCTtI4XidpKQ8vvarTQkf1qbr9GmIK/view?usp=sharing`

2.3.3 Plots of New Task Case

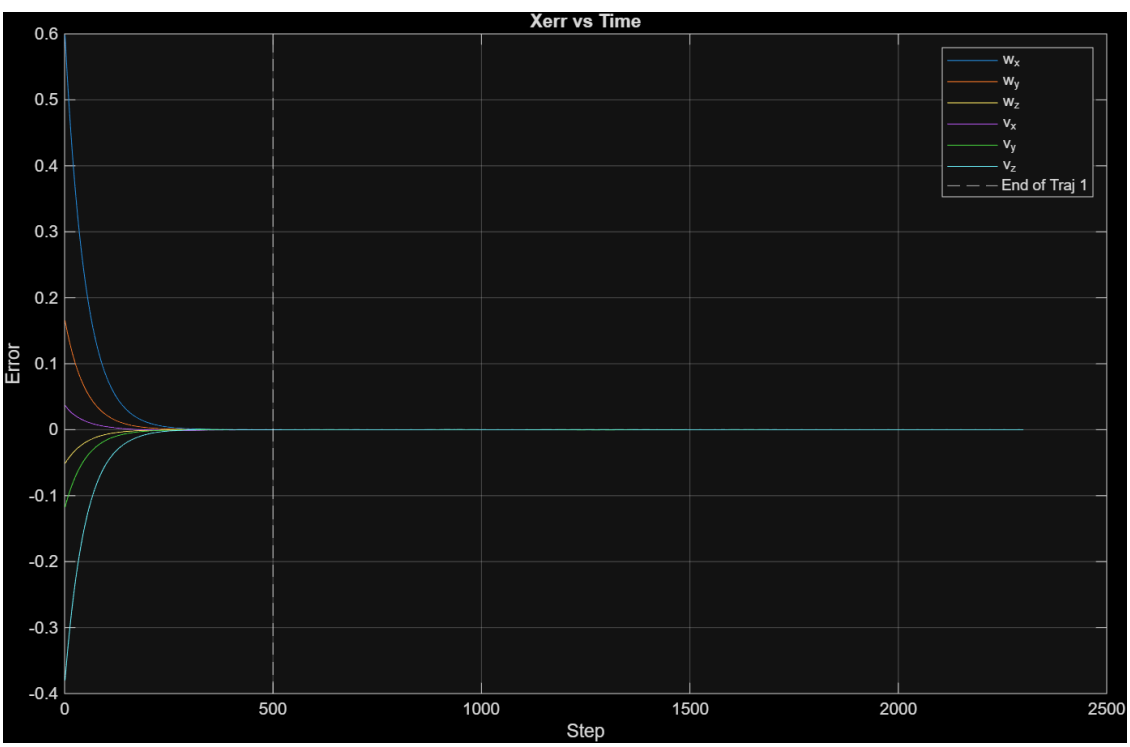

Figure 6: Xerr vs. Time for New Task Case

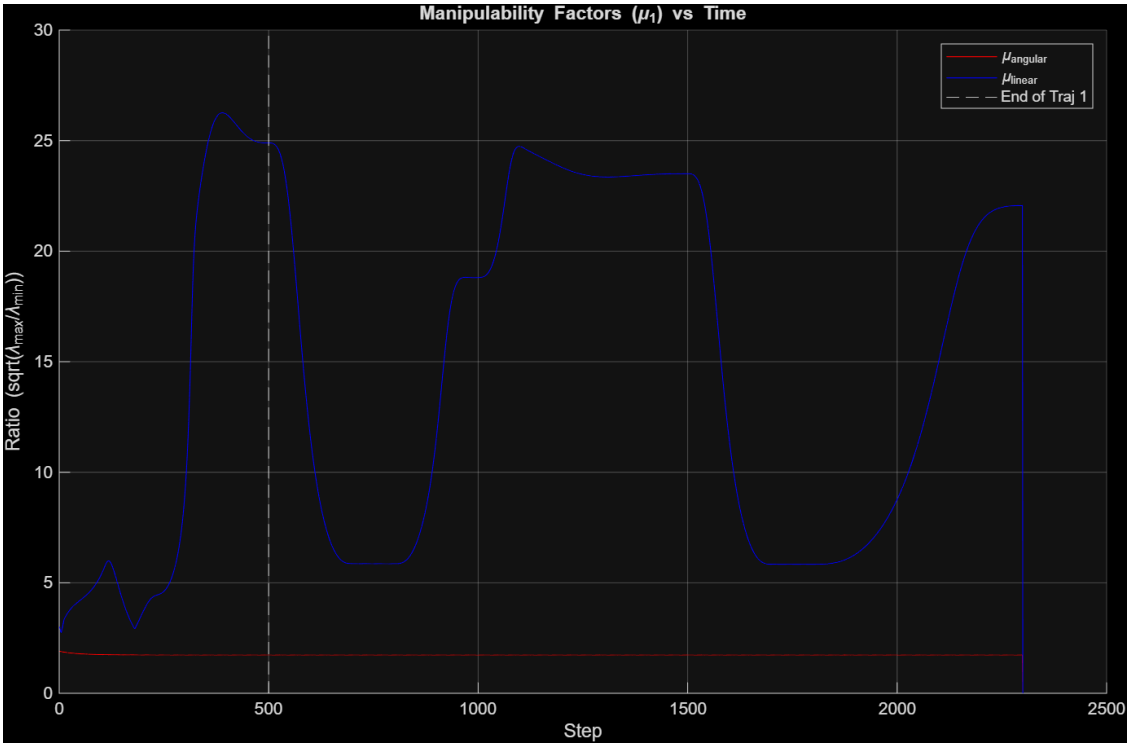

Figure 7: $u_1(A_w)$ and $u_1(A_v)$ vs. Time for New Task Case

3 Discussion

The primary advantage of an integral controller is its ability to eliminate steady-state error. With just a P controller, a condition can arise where the system settles at an undesired equilibrium point because the P controller's command is insufficient to overcome constant, unaccounted for forces like friction. The integral component accumulates the remaining error over time and provides the necessary push to overcome these offsets. However, as shown in Figure 3, the integral component has plenty of error to accumulate at the start. By the time the end-effector reaches the desired configuration, this accumulated error results in a velocity command that forces the system to overshoot. Since the simulation is "perfect," the absence of steady-state offset sources means that the P and feedforward components are mathematically sufficient to bring the equilibrium point to the desired configuration. As a result, the integral term primarily serves to create underdamped behavior in the system.

In all cases, the manipulability factors became large when the end-effector approached the end of trajectory one. This lines up with when the arm is outstretched and ready to pick up the cube. Being outstretched results in the system heading toward a singularity and losing the ability to move the end-effector in certain directions. On the other side to this, the manipulability factors were small when the arm was somewhat (but not fully)"folded" up, and so all joints had more freedom to move in various directions.

Being near the kinematic singularity also results in larger required joint velocities for small end-effector motions. When these commanded velocities exceed the magnitude of the joint limits, the robot can not keep up with the desired trajectory and this results in the increased error observed. On top of this, right after the cube is grasped, the desired trajectory

requires faster and more dynamic motion. The end-effector has to go from slowly grasping the cube to suddenly changing direction and moving to the next task. This results in higher joint velocities that can not be actuated and so the error increases. These influences become significantly more relevant as the maximum joint velocities are further limited.

Velocity control is realistic for tasks in which the robot is basically moving in free space. That is, there are no outside forces that the robot has to push against, the motors are sufficient to reliably track the commanded velocity, and all the robot has to do is track motions. For example, velocity control is realistic for moving the robot from one place to another in an unobstructed plane. Velocity control is unrealistic for situations where the robot has to interact with the outside environment and apply forces. Even something as simple as pressing the end-effector against a surface (cleaning a window, pressing a button, fitting a bolt into a nut, etc.) would not be reliably done with velocity control. The robot will simply try to follow the desired path, and may push until the end-effector is damaged, or the surface is damaged.

To implement torque control for the youBot, Feedback Control would require the actual and desired joint positions, velocities, and accelerations. It would also need the screw axes of all the joints at the home position in the space frame, the inertia matrices of all the links, pose of every link relative to the previous link in the chain at home position, the pose of the first link relative to the space frame at home position, and the constant gravity vector. Realistically, if the robot was to interact with the environment, it would also have some sort of force sensors on the end-effector. The MR function to use for this would be InverseDynamics.

Generative AI (ChatGPT) was utilized to help debug errors that appeared when trying to implement the cell array in TrajectoryGenerator.